

```
#Importing pandas library
import pandas as pd
```

```
#Loading the dataframe from a csv file using pandas
df = pd.read_csv("Churn.csv")
df.head()
```

[Show hidden output](#)

EXPLORATORY DATA ANALYSIS

```
#Exploring the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3333 entries, 0 to 3332
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Account_Length        3333 non-null   int64
1   Vmail_Message         3333 non-null   int64
2   Day_Mins              3333 non-null   float64
3   Eve_Mins              3333 non-null   float64
4   Night_Mins            3333 non-null   float64
5   Intl_Mins             3333 non-null   float64
6   CustServ_Calls        3333 non-null   int64
7   Churn                 3333 non-null   object
8   Intl_Plan             3333 non-null   object
9   Vmail_Plan            3333 non-null   object
10  Day_Calls             3333 non-null   int64
11  Day_Charge            3333 non-null   float64
12  Eve_Calls             3333 non-null   int64
13  Eve_Charge            3333 non-null   float64
14  Night_Calls           3333 non-null   int64
15  Night_Charge          3333 non-null   float64
16  Intl_Calls            3333 non-null   int64
17  Intl_Charge           3333 non-null   float64
18  State                 3333 non-null   object
19  Area_Code             3333 non-null   int64
20  Phone                 3333 non-null   object
dtypes: float64(8), int64(8), object(5)
memory usage: 546.9+ KB
```

There are total 21 features and 3333 observations in the dataset

```
#Exploring the dataset
df.describe()
```

	Account_Length	Vmail_Message	Day_Mins	Eve_Mins	Night_Mi
count	3333.000000	3333.000000	3333.000000	3333.000000	3333.0000
mean	101.064806	8.099010	179.775098	200.980348	200.8720
std	39.822106	13.688365	54.467389	50.713844	50.5738
min	1.000000	0.000000	0.000000	0.000000	23.2000
25%	74.000000	0.000000	143.700000	166.600000	167.0000
50%	101.000000	0.000000	179.400000	201.400000	201.2000
75%	127.000000	20.000000	216.400000	235.300000	235.3000
max	243.000000	51.000000	350.800000	363.700000	395.0000

*Continuous features such as Account Length, Day Minutes, Evening Minutes, and Night Minutes appear to follow approximately symmetric distributions, with mean values close to their medians. This suggests the absence of strong skewness.

Customer Service Calls shows a slightly skewed (right)distribution, with higher values associated with mean compared to median.

The dataset contains features with varying scales (e.g., minutes vs number of calls), which may require normalization for certain models.*

```
#Exploring Customer Churn
df["Churn"].value_counts()
```

	count
Churn	
no	2850
yes	483
dtype: int64	

Observation: *Imbalanced classes with non-churners being thrice of the churners*

SUMMARY STATISTICS FOR BOTH CLASSES

```
#Subsetting the data to investigate specific features
df_custserv_vmsg = df[['Churn', 'CustServ_Calls', 'Vmail_Message']]
df_custserv_vmsg.head()
```

	Churn	CustServ_Calls	Vmail_Message
0	no	1	25
1	no	1	26
2	no	0	0
3	no	2	0
4	no	3	0

```
#Mean for Churners vs Non-Churners
df_custserv_vmsg.groupby(["Churn"]).mean()
```

	CustServ_Calls	Vmail_Message
Churn		
no	1.449825	8.604561
yes	2.229814	5.115942

```
#Standard deviation for Churners vs Non-Churners
df_custserv_vmsg.groupby(["Churn"]).std()
```

	CustServ_Calls	Vmail_Message
Churn		
no	1.163883	13.913125
yes	1.853275	11.860138

Customer Service Calls shows a slightly skewed distribution, with higher values associated with churned customers, indicating its potential importance as a predictor.

```
#Churn vs Churners by State  
df.groupby("State")["Churn"].value_counts()
```

		count
State	Churn	
AK	no	49
	yes	3
AL	no	72
	yes	8
AR	no	44
...	...	...
WI	yes	7
WV	no	96
	yes	10
WY	no	68
	yes	9

102 rows x 1 columns

dtype: int64

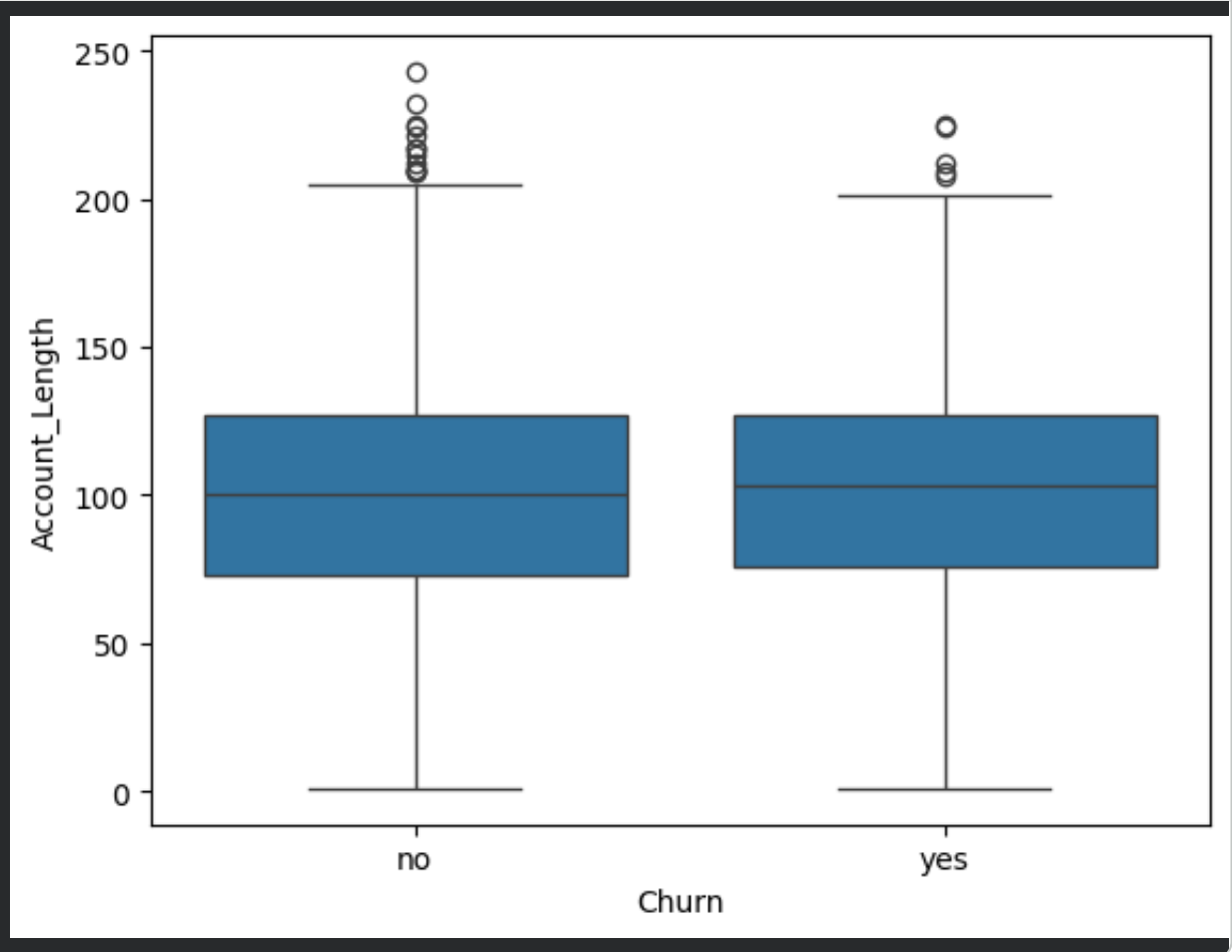
```
#Statewise churn counts comparison  
df.groupby("State")["Churn"].value_counts()[["CA","CO","AL"]]
```

		count
State	Churn	
CA	no	25
	yes	9
CO	no	57
	yes	9
AL	no	72
	yes	8

dtype: int64

VISUALISING THE DATA

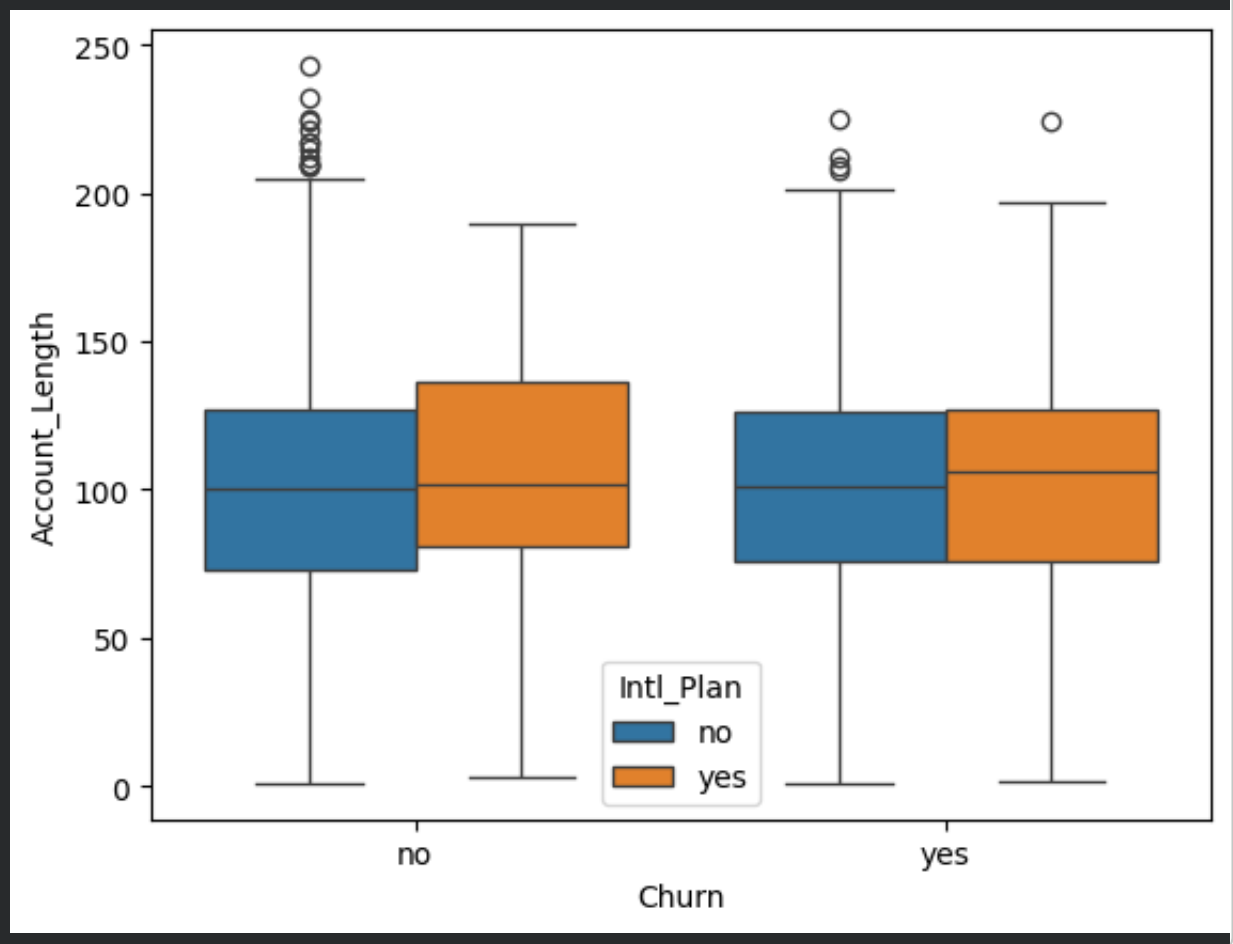
```
#Importing data visualisation libraries – Matplotlib & Seaborn
import matplotlib.pyplot as plt
import seaborn as sns
sns.boxplot(x= 'Churn',
            y= 'Account_Length',
            data = df)
plt.show()
```



Observation: *No difference in account length for Churners vs Non-Churners.*

```
# Adding a third variable to understand it's impact on the Churn an
import matplotlib.pyplot as plt
import seaborn as sns
sns.boxplot(x= 'Churn',
            y= 'Account_Length',
            data = df,
            hue='Intl_Plan'
            )

plt.show()
```



Observation: *No impact of International Plan*

```
#Distribution plot the features(Day_Mins,Eve_Mins, Night_Mins, Intl  
sns.distplot(df['Day_Mins'])  
plt.show()
```

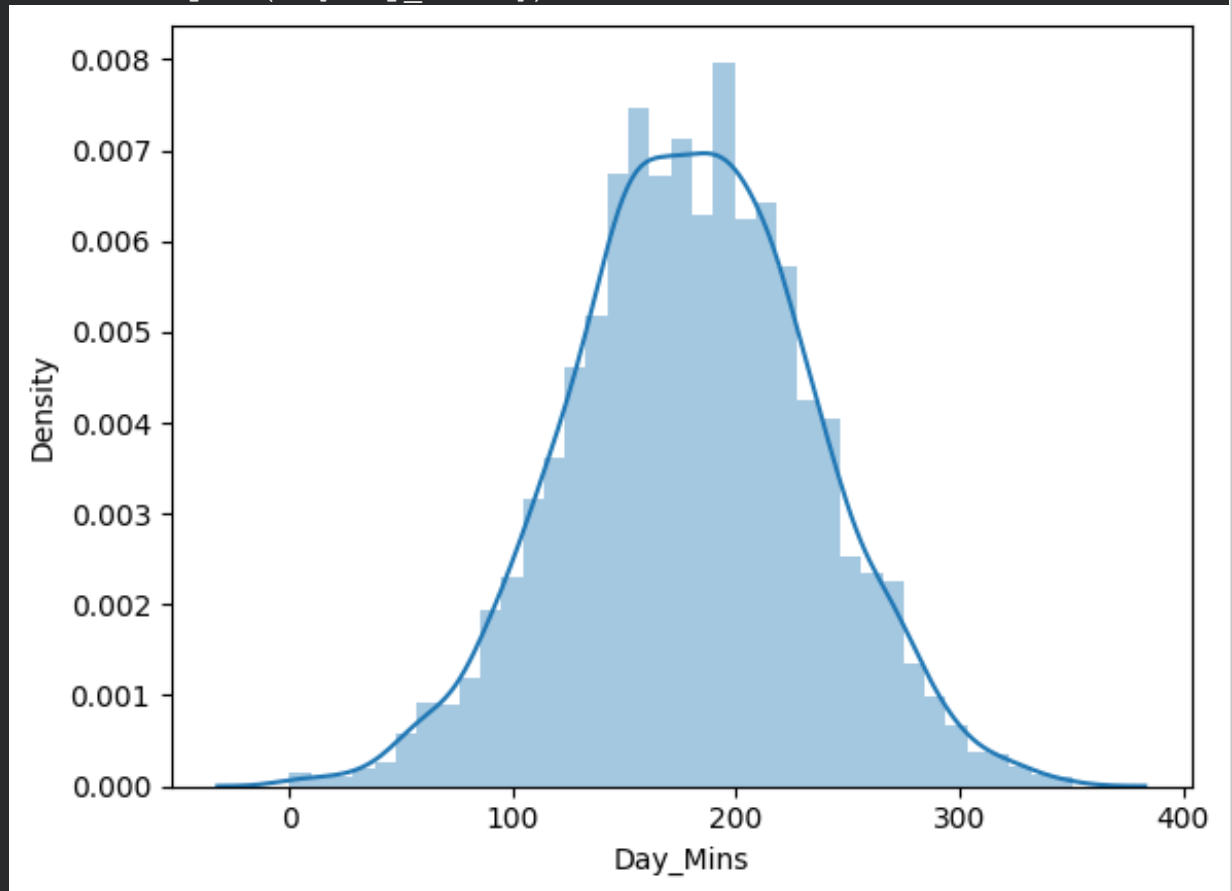
```
/tmp/ipykernel_10723/3591231309.py:2: UserWarning:
```

```
`distplot` is a deprecated function and will be removed in seaborn v
```

```
Please adapt your code to use either `displot` (a figure-level funct  
similar flexibility) or `histplot` (an axes-level function for histo
```

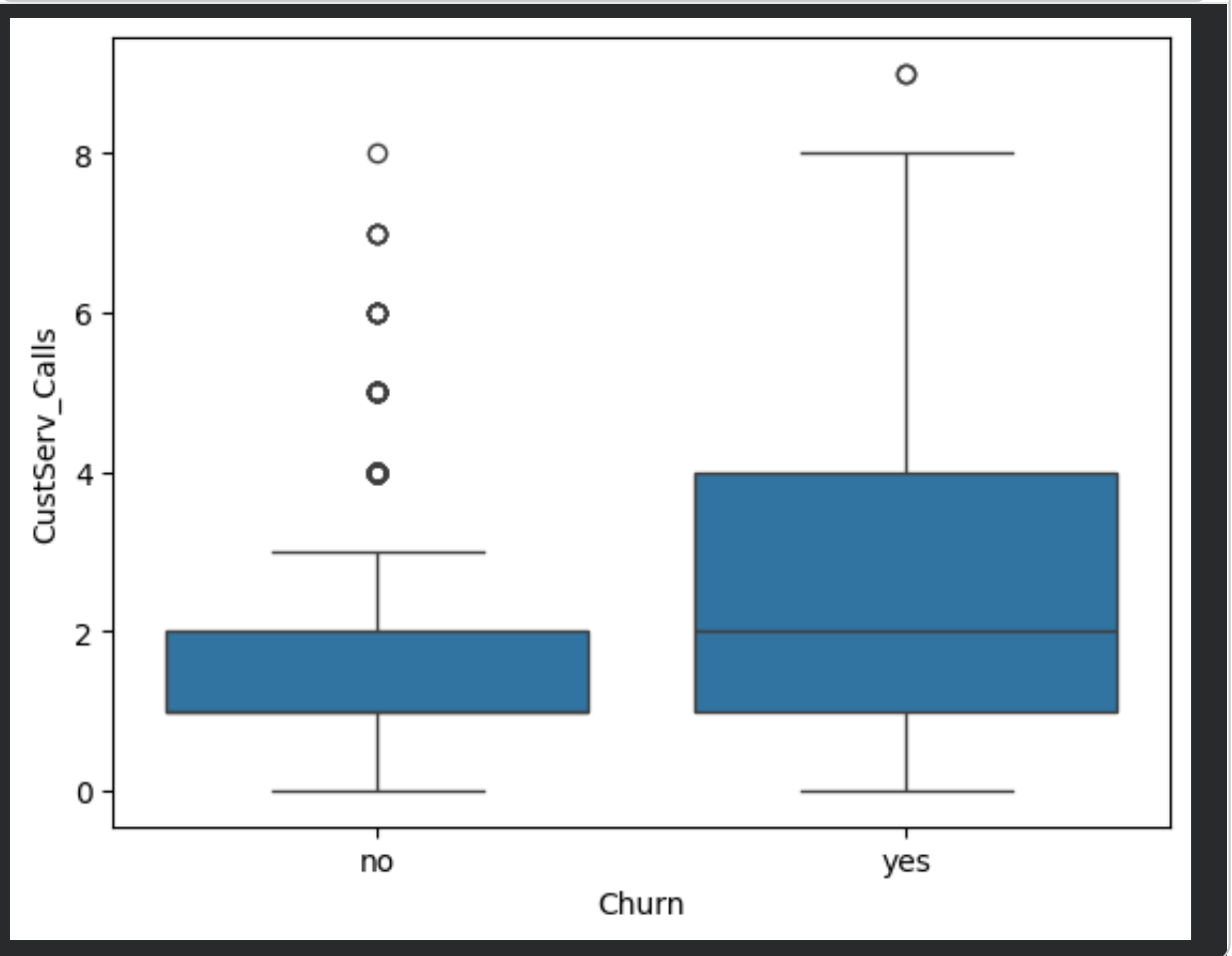
```
For a guide to updating your code to use the new functions, please s  
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751
```

```
sns.distplot(df['Day_Mins'])
```



Observation: *All(Day_Mins,Eve_Mins, Night_Mins, Intl_mins) of these features appear to be well approximated by the normal distribution. If this were not the case,we would have to consider applying a feature transformation of some kind.*

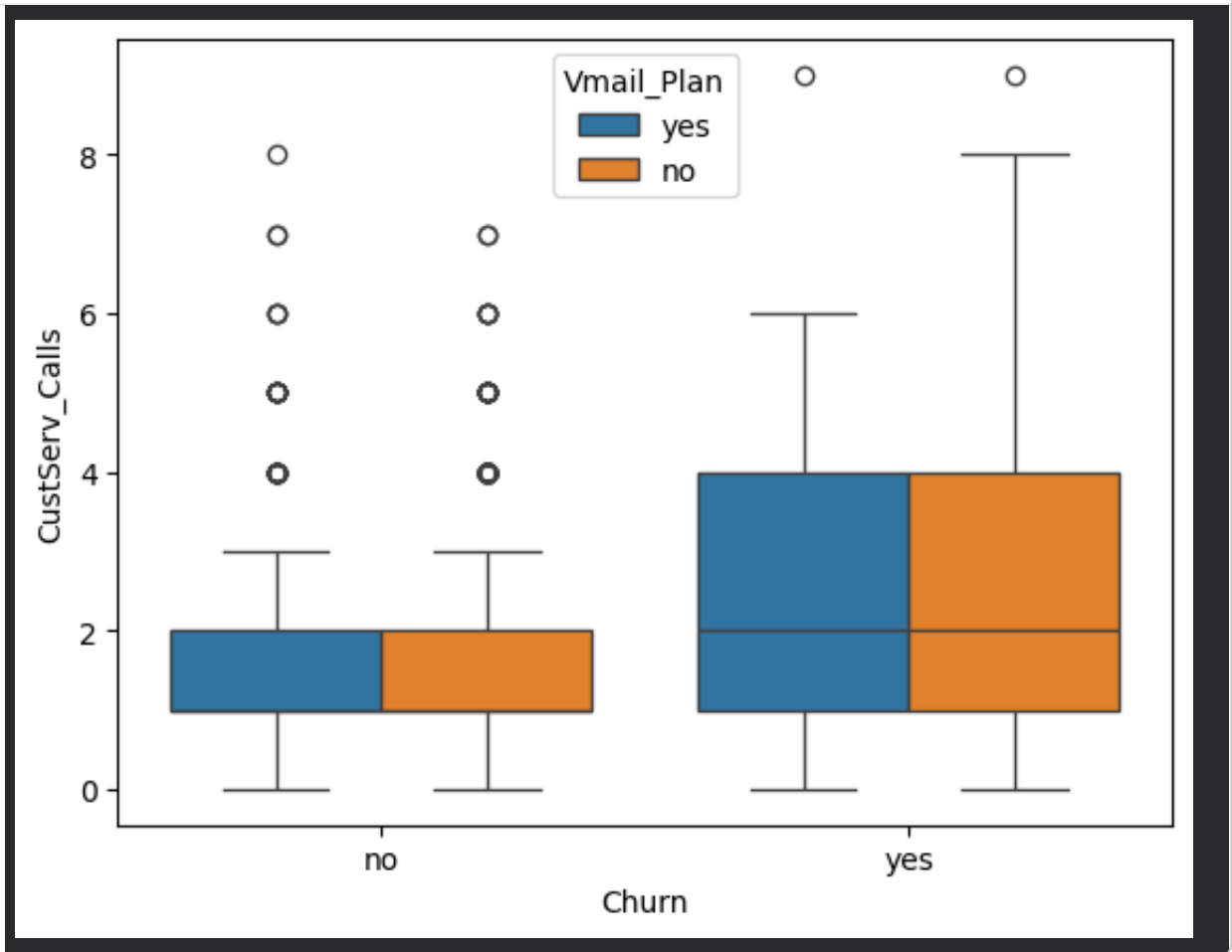
```
#Lets investigate the impact of Customer Service Calls on the Churn
sns.boxplot(x='Churn',
            y='CustServ_Calls',
            data=df)
plt.show()
```



Observation: # The box plot reveals that the Customer Service Calls feature has a noticeable difference (much higher) for churners compared to non-churners

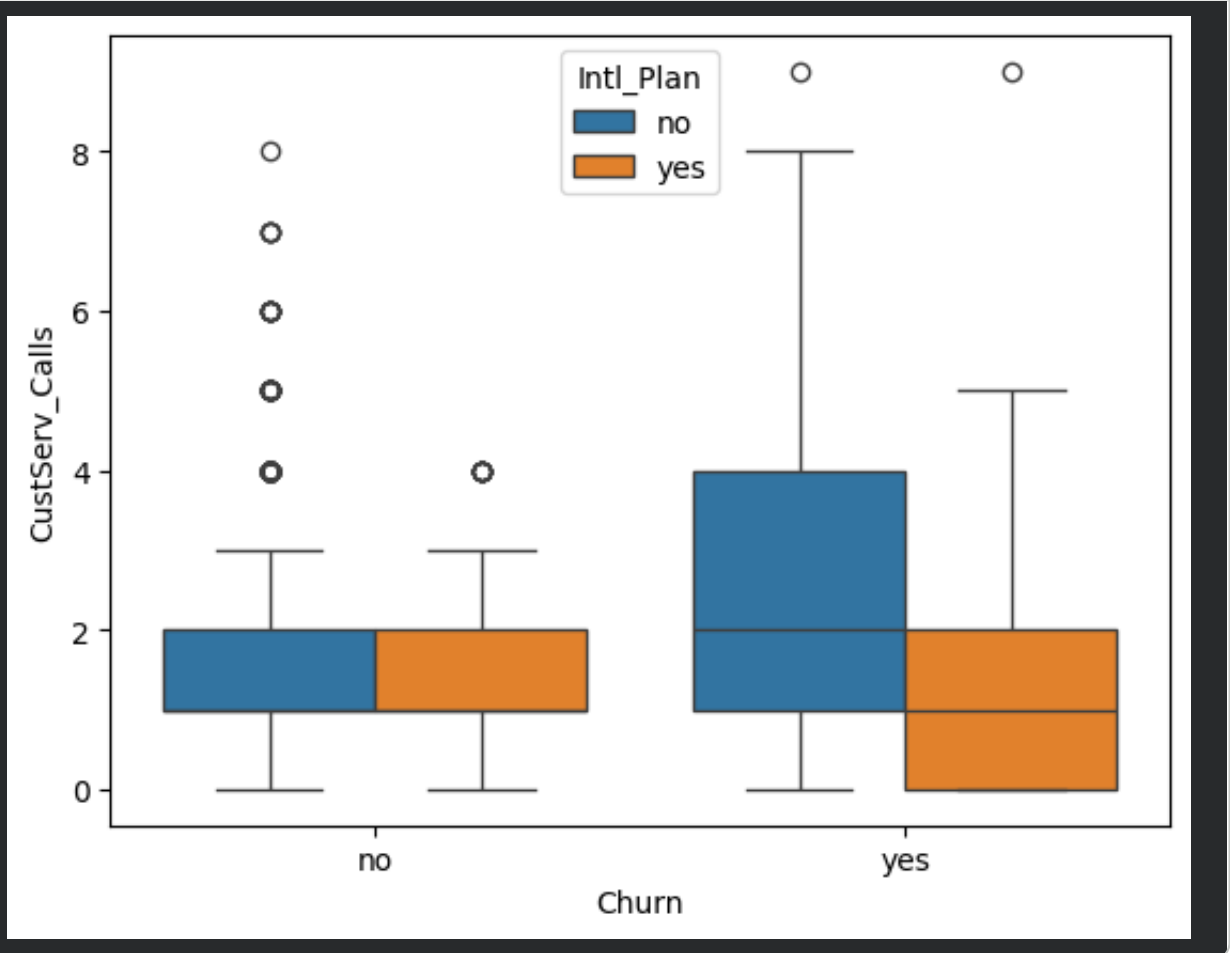
```
#Impact of Customer Vmail_Plan on Churn
import matplotlib.pyplot as plt
import seaborn as sns
sns.boxplot(x='Churn',
            y='CustServ_Calls',
            data=df,
            hue='Vmail_Plan')

plt.show()
```



Observation:** On plotting the box plot, we see that there is not much impact of Vmail_Plan on Churners/Non-churners

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.boxplot(x='Churn',
            y='CustServ_Calls',
            data=df,
            hue='Intl_Plan')
plt.show()
```



Observations: *Interestingly on plotting the box plot, we see that customers who do churn end up leaving due to more service calls unless they are on an International plan in which case they make less customer service calls.

Since both Blue and Orange boxes on the right are bigger than on left, it is clear that customers who make service calls churn more but on adding the International plan feature, we see that the box on the extreme right is shifted to a lower value reflecting that customers with Intl plan are making fewer Customer Service calls.

DATA PREPROCESSING

```
#Encoding binary features
```

```
df['Churn'] = df['Churn'].replace({'yes':1,'no':0})
df['Vmail_Plan'] = df['Vmail_Plan'].replace({'yes':1,'no':0})
df['Intl_Plan'] = df['Intl_Plan'].replace({'yes':1,'no':0})
df.head()
```

```
/tmp/ipykernel_10723/4099571106.py:2: FutureWarning: Downcasting beh
df['Churn'] = df['Churn'].replace({'yes':1,'no':0})
/tmp/ipykernel_10723/4099571106.py:3: FutureWarning: Downcasting beh
df['Vmail_Plan'] = df['Vmail_Plan'].replace({'yes':1,'no':0})
/tmp/ipykernel_10723/4099571106.py:4: FutureWarning: Downcasting beh
df['Intl_Plan'] = df['Intl_Plan'].replace({'yes':1,'no':0})
```

	Account_Length	Vmail_Message	Day_Mins	Eve_Mins	Night_Mins	Intl
0	128	25	265.1	197.4	244.7	
1	107	26	161.6	195.5	254.4	
2	137	0	243.4	121.2	162.6	
3	84	0	299.4	61.9	196.9	
4	75	0	166.7	148.3	186.9	

5 rows × 21 columns

```
#One hot encoding for categorical feature - State
```

```
df_state = pd.get_dummies(df['State'])
df_state.head()
```

	AK	AL	AR	AZ	CA	CO	CT	DC	DE	FL	...	S
0	False	False	False	False	False	False	False	False	False	False	...	False
1	False	False	False	False	False	False	False	False	False	False	...	False
2	False	False	False	False	False	False	False	False	False	False	...	False
3	False	False	False	False	False	False	False	False	False	False	...	False
4	False	False	False	False	False	False	False	False	False	False	...	False

5 rows × 51 columns

```
#Exploring the scale of features - Intl_calls (Feature Scaling)
df['Intl_Calls'].describe()
```

```
Intl_Calls
count    3333.000000
mean      4.479448
std       2.461214
min       0.000000
25%       3.000000
50%       4.000000
75%       6.000000
max      20.000000
```

```
dtype: float64
```

```
#Exploring the scale of features - Night_Mins (Feature Scaling)
df['Night_Mins'].describe()
```

```
Night_Mins
count    3333.000000
mean     200.872037
std      50.573847
min      23.200000
25%     167.000000
50%     201.200000
75%     235.300000
max     395.000000
```

```
dtype: float64
```

```
#Feature Engineering – Average Night Calls = diving Night Minutes b
#Creating the new feature
df['Avg_Night_Calls'] = df['Night_Mins']/df['Night_Calls']
df['Avg_Night_Calls'].head()
```

```

  Avg_Night_Calls
0      2.689011
1      2.469903
2      1.563462
3      2.212360
4      1.544628
```

```
dtype: float64
```

```
#Feature Engineering – Dropping unnecessary features
cols_to_drop =df[['State', 'Area_Code', 'Phone', 'Churn']]
cols_to_drop.head()
df_features = df.drop(cols_to_drop,axis=1)
df_features.head()
```

```

  Account_Length  Vmail_Message  Day_Mins  Eve_Mins  Night_Mins  Int.
0           128             25    265.1    197.4    244.7
1           107             26    161.6    195.5    254.4
2           137              0    243.4    121.2    162.6
3            84              0    299.4     61.9    196.9
4            75              0    166.7    148.3    186.9
```

```
df_features.corr()
```

	Account_Length	Vmail_Message	Day_Mins	Eve_Mins	Night_Mins
Account_Length	1.000000	-0.004628	0.006216	-0.006757	-0.008955
Vmail_Message	-0.004628	1.000000	0.000778	0.017562	0.007681
Day_Mins	0.006216	0.000778	1.000000	0.007043	0.004323
Eve_Mins	-0.006757	0.017562	0.007043	1.000000	-0.012584
Night_Mins	-0.008955	0.007681	0.004323	-0.012584	1.000000
Intl_Mins	0.009514	0.002856	-0.010155	-0.011035	0.002856
CustServ_Calls	-0.003796	-0.013263	-0.013423	-0.012985	-0.013263
Intl_Plan	0.024735	0.008745	0.049396	0.019100	0.008745
Vmail_Plan	0.002918	0.956927	-0.001684	0.021545	0.002918
Day_Calls	0.038470	-0.009548	0.006750	-0.021451	-0.009548
Day_Charge	0.006214	0.000776	1.000000	0.007050	0.000776
Eve_Calls	0.019260	-0.005864	0.015769	-0.011430	-0.005864
Eve_Charge	-0.006745	0.017578	0.007029	1.000000	0.017578
Night_Calls	-0.013176	0.007123	0.022972	0.007586	0.007123
Night_Charge	-0.008960	0.007663	0.004300	-0.012593	0.007663
Intl_Calls	0.020661	0.013957	0.008033	0.002541	0.013957
Intl_Charge	0.009546	0.002884	-0.010092	-0.011067	0.002884
Avg_Night_Calls	-0.004067	0.003061	-0.011777	-0.011740	0.003061

Highly correlated features, can be dropped as they provide no additional information to the model - Day_Mins & Day_Charge, Eve_Mins & Eve_Charge, Night_Mins & Night_Charge, Intl_Mins & intl_Charge.

```
#Feature Scaling
from sklearn.preprocessing import StandardScaler #Importing Standar
df_scaled = StandardScaler().fit_transform(df_features)#Scale df us
df_for_md = pd.DataFrame(df_scaled,columns=df_features.columns)#Add
df_for_md.head()
```

	Account_Length	Vmail_Message	Day_Mins	Eve_Mins	Night_Mins	Int.
0	0.676489	1.234883	1.566767	-0.070610	0.866743	-0.0
1	0.149065	1.307948	-0.333738	-0.108080	1.058571	1.0
2	0.902529	-0.591760	1.168304	-1.573383	-0.756869	0.0
3	-0.428590	-0.591760	2.196596	-2.742865	-0.078551	-1.0
4	-0.654629	-0.591760	-0.240090	-1.038932	-0.276311	-0.0

MODEL PREDICTION Goal: to predict whether the customer will Churn or not

Target Variable: 'Churn'

Techniques: Supervised Machine Learning

Model Selction: 1.Logistic regression, 2.Decision tree, 3.Support vector machines, 4.Random forests 5.XGBOOST

LOGISTIC REGRESSION MODEL + EVALUATION + METRICS + AUC

```
#Importing train test split
from sklearn.model_selection import train_test_split
#Importing LogisticRegression
from sklearn.linear_model import LogisticRegression

#Create training and testing sets
X_train,X_test,y_train,y_test = train_test_split(df_for_md, df['Chu

# Instantiating the classifier
clf=LogisticRegression()

# Fitting into the training data
clf.fit(X_train,y_train)

# Predicting the label for X_test
y_pred = clf.predict(X_test)
```

```
#Accuracy Score
Acc = clf.score(X_test,y_test)
print(f"Accuracy: {Acc:.2f}")

#Precision Score
from sklearn.metrics import precision_score
print(f"Precision: {precision_score(y_test,y_pred):.2f}")

#Recall Score
from sklearn.metrics import recall_score
print(f"Recall: {recall_score(y_test,y_pred):.2f}")

#F1 score
from sklearn.metrics import f1_score
print(f"F1Score:{f1_score(y_test,y_pred):.2f}")

#Other metrics – AOC, ROC

#Generate probabilities
y_pred_prob = clf.predict_proba(X_test)[:,-1]

#Importing roc_curve
from sklearn.metrics import roc_curve

#Calculating roc metrics
fpr,tpr,thresholds = roc_curve(y_test,y_pred_prob)

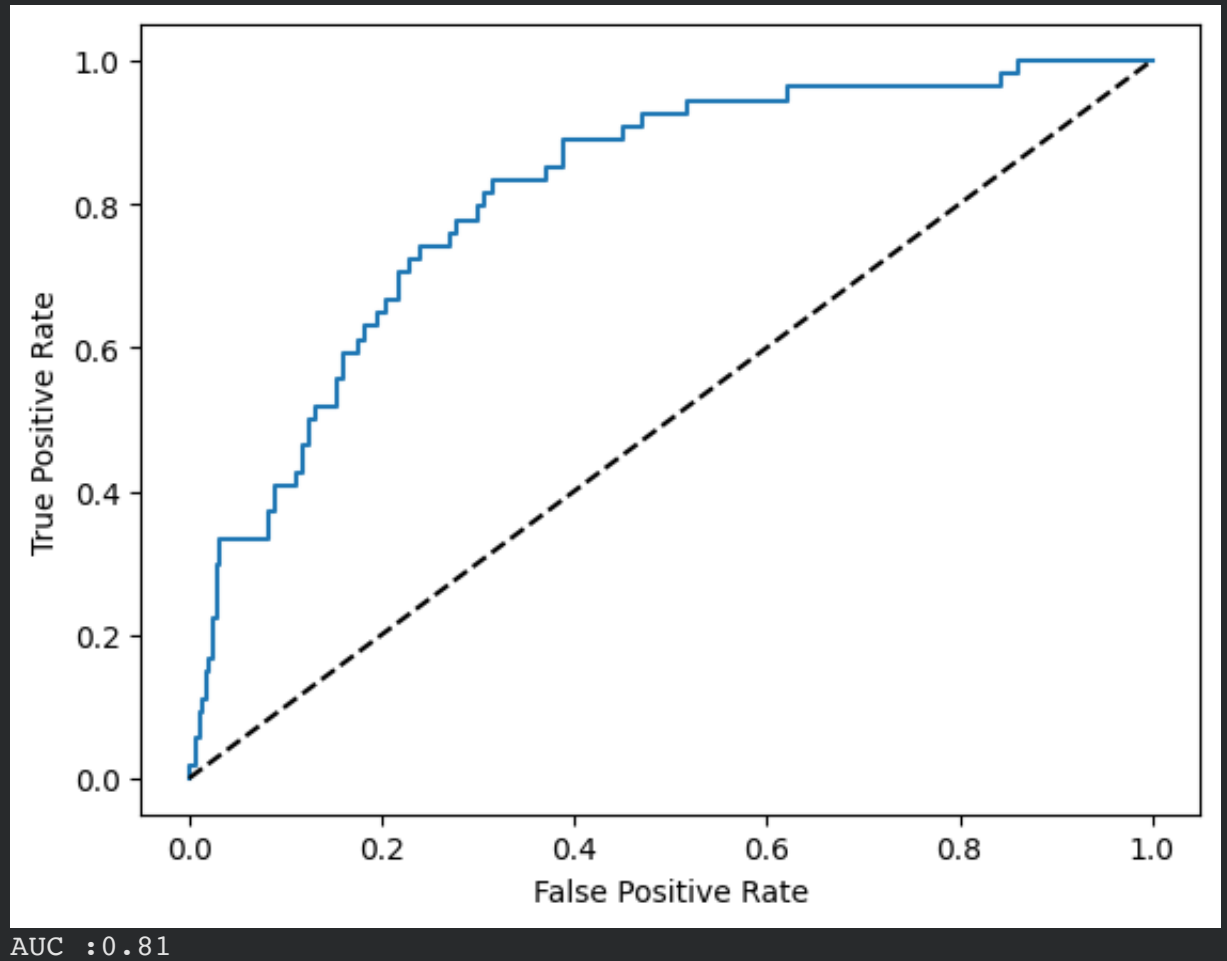
#Plotting the ROC curve
import matplotlib.pyplot as plt
plt.plot(fpr,tpr)

#Add labels and diagonal line(thresholds)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.plot([0, 1], [0, 1], "k--")
plt.show()

# Importing roc_auc_score
from sklearn.metrics import roc_auc_score

# Printing the AUC(Area Under the Curve)
print(f"AUC :{roc_auc_score(y_test, y_pred_prob):.2f}")
```

```
Accuracy: 0.85  
Precision: 0.59  
Recall: 0.19  
F1Score:0.28
```



Observations *Class Imbalance and Overfitting beacuse of the high Accuracy and LowF1 score.*

High Accuracy is misleading because Churn being the minority class, the model can predict mostly non - churners and get high accuracy but miss the actual churn cases hence low recall and F1.

DECISION TREE MODEL + EVALUATION

```
#Importing Libraries
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

#Creating training and test sets
X_train,X_test,y_train,y_test = train_test_split(df_for_md, df['Chu

#Instantiating the classifier
Clf=DecisionTreeClassifier()

#Fitting into the training data
Clf.fit(X_train,y_train)

#Predicting the label for X_test
y_pred = clf.predict(X_test)

Acc = clf.score(X_test,y_test) #Accuracy Score
print(f"Accuracy: {Acc:.2f}")

from sklearn.metrics import precision_score
print(f"Precision: {precision_score(y_test,y_pred):.2f}") #Precisio

from sklearn.metrics import recall_score
print(f"Recall: {recall_score(y_test,y_pred):.2f}") #Recall

from sklearn.metrics import f1_score
print(f"F1Score:{f1_score(y_test,y_pred):.2f}")#F1 score
```

```
Accuracy: 0.86
Precision: 0.61
Recall: 0.19
F1Score:0.29
```

SVM MODEL + EVALUATION

```
#Importing Libraries
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

#Creating training and test sets
X_train,X_test,y_train,y_test = train_test_split(df_for_md, df['Chu

#Instantiating the classifier
Clf=SVC()

#Fitting into the training data
Clf.fit(X_train,y_train)

#Predicting the label for X_test
y_pred = clf.predict(X_test)

Acc = clf.score(X_test,y_test) #Accuracy Score
print(f"Accuracy: {Acc:.2f}")

from sklearn.metrics import precision_score
print(f"Precision: {precision_score(y_test,y_pred):.2f}") #Precisio

from sklearn.metrics import recall_score
print(f"Recall: {recall_score(y_test,y_pred):.2f}") #Recall

from sklearn.metrics import f1_score
print(f"F1Score:{f1_score(y_test,y_pred):.2f}")#F1 score
```

```
Accuracy: 0.86
Precision: 0.61
Recall: 0.19
F1Score:0.29
```

RANDOM FOREST MODEL + EVALUATION + MODEL TUNING

```
#Importing Libraries
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

#Creating training and test sets
X_train,X_test,y_train,y_test = train_test_split(df_for_md, df['Chu

#Instantiating the classifier
Clf=RandomForestClassifier()

#Fitting into the training data
Clf.fit(X_train,y_train)

#Predicting the label for X_test
y_pred = clf.predict(X_test)

Acc = clf.score(X_test,y_test) #Accuracy Score
print(f"Accuracy: {Acc:.2f}")

from sklearn.metrics import precision_score
print(f"Precision: {precision_score(y_test,y_pred):.2f}") #Precisio

from sklearn.metrics import recall_score
print(f"Recall: {recall_score(y_test,y_pred):.2f}") #Recall

from sklearn.metrics import f1_score
print(f"F1Score:{f1_score(y_test,y_pred):.2f}")#F1 score
```

```
Accuracy: 0.86
Precision: 0.61
Recall: 0.19
F1Score:0.29
```

MODEL TUNING - RandomizedSearchCV

```
# Importing RandomizedSearchCV
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier
from scipy.stats import randint

# Creating the hyperparameter grid
param_grid = {"max_depth": [3, None],
              "max_features": randint(1, 11),
              "bootstrap": [True, False],
              "criterion": ["gini", "entropy"]}

# Instantiate the classifier
clf = RandomForestClassifier()

# Calling the RandomizedSearchCV
grid_search = RandomizedSearchCV(clf, param_grid, cv=3)

# Fitting the model
grid_search.fit(df_for_md, df['Churn'])

# Printing the best hyperparameters
print(grid_search.best_params_)
```

```
{'bootstrap': False, 'criterion': 'entropy', 'max_depth': None, 'max
```

```
best_rf = grid_search.best_estimator_
y_pred_rf = best_rf.predict(X_test)

from sklearn.metrics import accuracy_score, precision_score, recall

print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_pred):.2f}")
print(f"Recall: {recall_score(y_test, y_pred):.2f}")
print(f"F1 Score: {f1_score(y_test, y_pred):.2f}")
```

```
Accuracy: 0.86
Precision: 0.61
Recall: 0.19
F1 Score: 0.29
```

NEW IMPROVED MODEL TUNING + EVALUATION + METRICS

```
#Took the best parameters from tuning and manually recreated the model
#Added improvements like max_depth = 10, min_samples_split=10 etc

#Instantiating the classifier with the best hyperparameters
clf = RandomForestClassifier(
    n_estimators=200,
    criterion='entropy',
    max_depth=10, #To control overfitting
    max_features=5,
    min_samples_split=10,
    min_samples_leaf=5,
    bootstrap=True,
    class_weight='balanced',
    random_state=42
)

#Fitting the training set
clf.fit(X_train, y_train)

#Predicting the label for test set
y_pred = clf.predict(X_test)

from sklearn.metrics import accuracy_score, precision_score, recall_score

print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_pred):.2f}")
print(f"Recall: {recall_score(y_test, y_pred):.2f}")
print(f"F1 Score: {f1_score(y_test, y_pred):.2f}")
```

```
Accuracy: 0.96
Precision: 0.87
Recall: 0.82
F1 Score: 0.85
```

```
#Other metrics – AUC, ROC
```

```
#Generate probabilities
y_pred_prob = clf.predict_proba(X_test)[:,1]

from sklearn.metrics import roc_curve

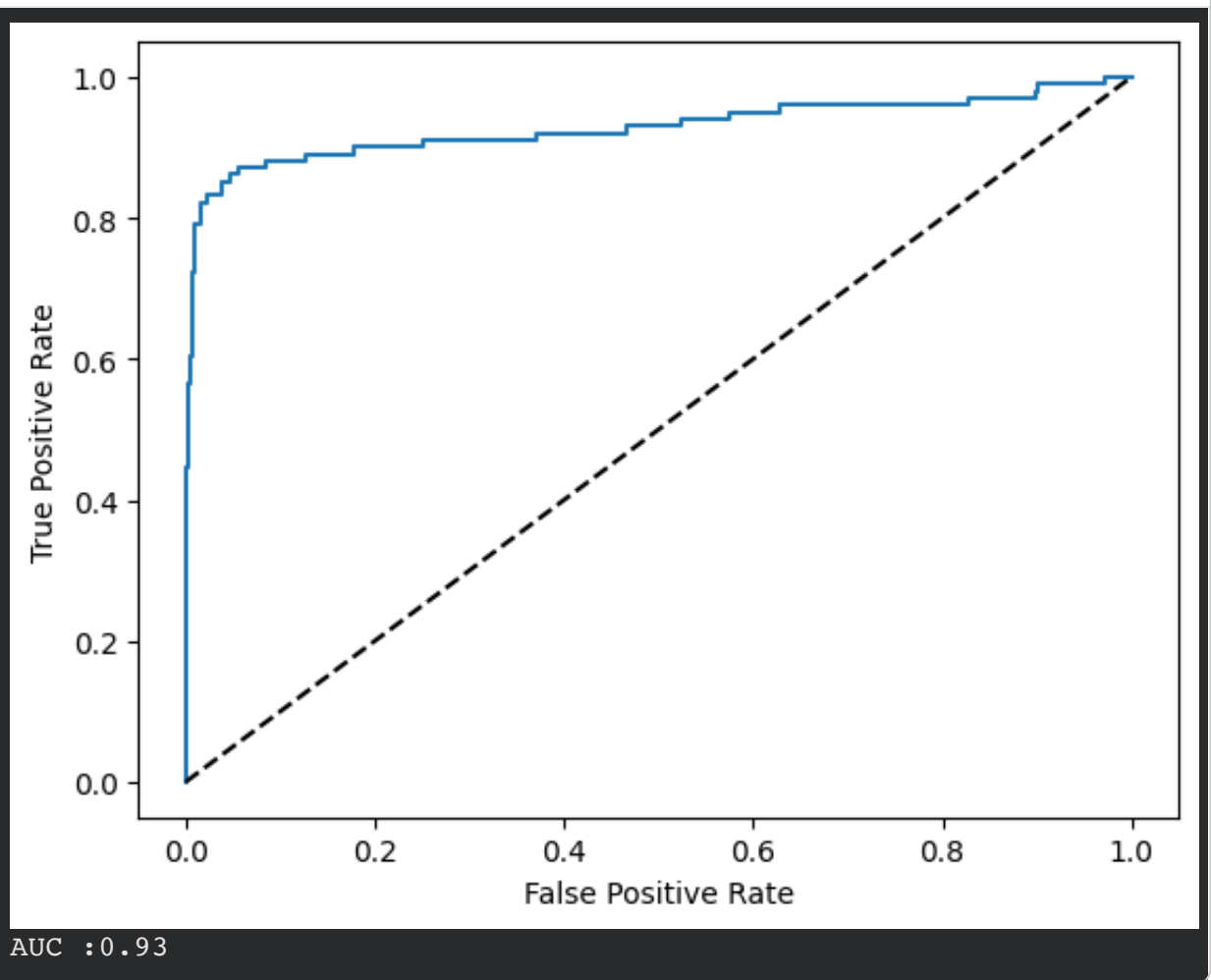
#Calculate roc metrics
fpr,tpr,thresholds = roc_curve(y_test,y_pred_prob)

#Plot the ROC curve
import matplotlib.pyplot as plt
plt.plot(fpr,tpr)
```

```
#Add labels and diagonal line(thresholds)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.plot([0, 1], [0, 1], "k--")
plt.show()

# Import roc_auc_score
from sklearn.metrics import roc_auc_score

# Print the AUC(Area Under the Curve)
print(f"AUC :{roc_auc_score(y_test, y_pred_prob):.2f}")
```



OBSERVATION *The model drastically improves by tuning the hyperparameters. The original method defaults to Deep tress (max_depth= None), no class balancing and several correlated features which leads to memorization of training data and poor generalisation*

To build the better model, we used tuned parameters, added class balancing and controlled overfitting by (max_depth=10,min_samples_split=10,min_samples_leaf=5).

VISUALISING THE FEATURE IMPORTANCE

```
from sklearn.ensemble import RandomForestClassifier
import numpy as np
import matplotlib.pyplot as plt

#Instintiating the classifier
clf = RandomForestClassifier()

#Fitting the model
clf.fit(df_for_md, df['Churn'])

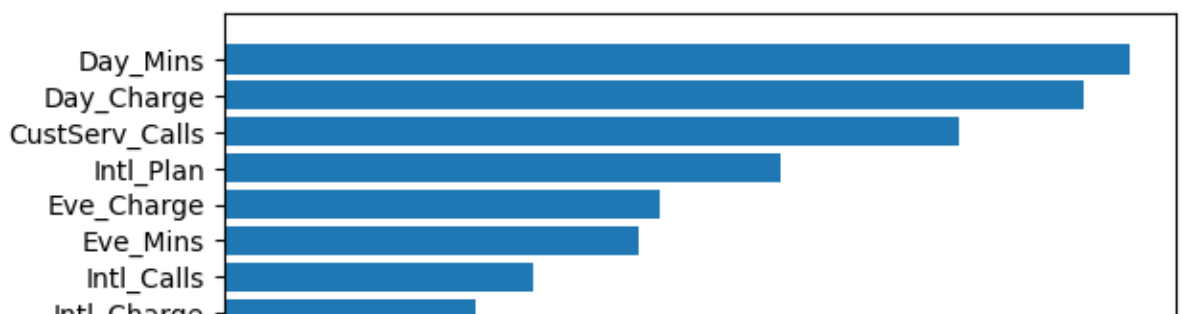
# Calculating feature importances
importances = clf.feature_importances_

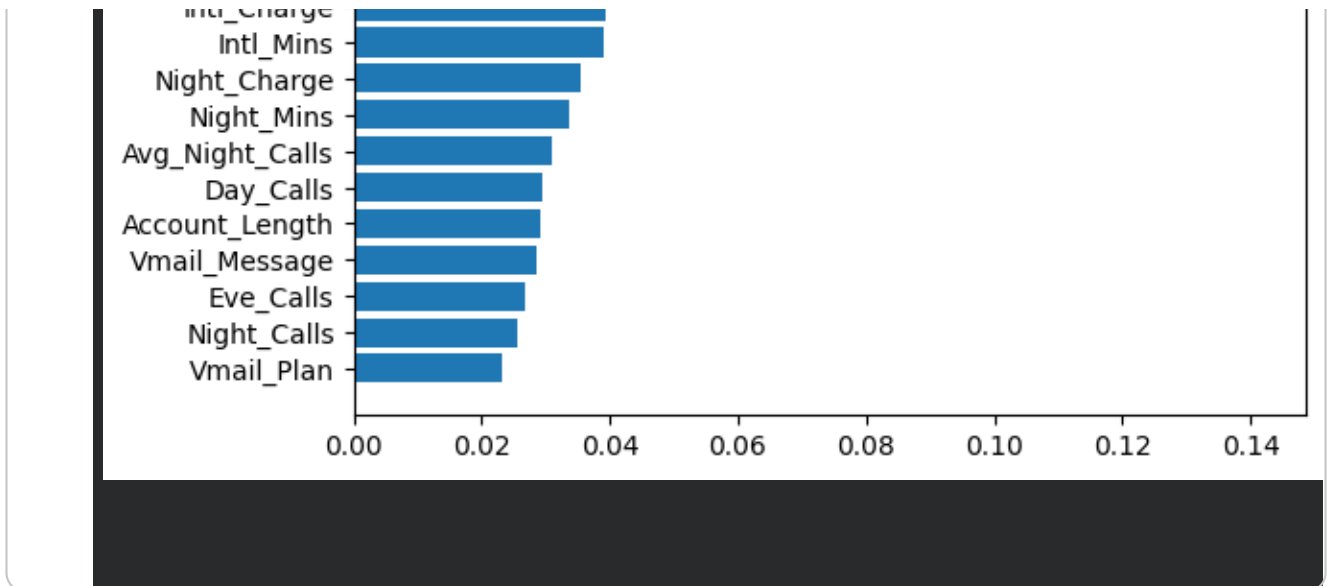
# Sorting importances
sorted_index = np.argsort(importances)

# Creating labels
labels = df_for_md.columns[sorted_index]

# Clearing current plot
plt.clf()

# Creating plot
plt.barh(range(df_for_md.shape[1]), importances[sorted_index], tick
plt.show())
```





Observation: *Day usage and customer service interactions are the strongest drivers of churn, with redundancy observed between Day_Mins and Day_Charge due to their direct relationship*

BUILDING XGBOOST BASELINE MODEL + MODEL EVALUATION

```
#Importing libraries
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall

xgb = XGBClassifier(eval_metric='logloss', random_state=42)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(df_for_md, df['

#Fitting the training set
xgb.fit(X_train, y_train)

#Predicting the label for test set
y_pred = xgb.predict(X_test)

#Model Evaluation
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_pred):.2f}")
print(f"Recall: {recall_score(y_test, y_pred):.2f}")
print(f"F1 Score: {f1_score(y_test, y_pred):.2f}")
```

```
Accuracy: 0.95
Precision: 0.94
Recall: 0.74
F1 Score: 0.83
```

XGBOOST MODEL TUNING - RandomizedSearchCV

```
# Importing RandomizedSearchCV
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform

# Creating the hyperparameter grid
param_grid = {
    'n_estimators': randint(100, 300),
    'max_depth': randint(3, 10),
    'learning_rate': uniform(0.01, 0.2),
    'subsample': uniform(0.6, 0.4),
    'colsample_bytree': uniform(0.6, 0.4)
}

# Instantiating the classifier
xgb = XGBClassifier(
    scale_pos_weight=2850/483,
    eval_metric='logloss',
    random_state=42
)

# Calling the RandomizedSearchCV
random_search = RandomizedSearchCV(
    xgb,
    param_grid,
    n_iter=20,
    scoring='f1',    # 🔥 VERY IMPORTANT
    cv=3,
    random_state=42
)

#Fitting the model
random_search.fit(X_train, y_train)

#Printing the best parameters
print(random_search.best_params_)

{'colsample_bytree': np.float64(0.9140703845572055), 'learning_rate'
```

XGBOOST MODEL EVALUATION

```
best_xgb = random_search.best_estimator_  
  
#Predicting the label for test set  
y_pred = best_xgb.predict(X_test)  
  
from sklearn.metrics import accuracy_score, precision_score, recall  
  
#Model Evaluation  
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")  
print(f"Precision: {precision_score(y_test, y_pred):.2f}")  
print(f"Recall: {recall_score(y_test, y_pred):.2f}")  
print(f"F1 Score: {f1_score(y_test, y_pred):.2f}")
```

```
Accuracy: 0.96  
Precision: 0.87  
Recall: 0.84  
F1 Score: 0.85
```

After hyperparameter tuning, XGBoost achieved comparable performance to Random Forest improved model, with a higher recall (0.84) and equal F1-score (0.85). Since churn prediction prioritizes identifying at-risk customers, the tuned XGBoost model is selected as the final model.

